

Robot-arm-task-rewrite-for-ai-delegation

****Test script/test automation**

Constraints:

- Use python
- Create a new file for every class, and a single files for utilities functions
- The whole solution seems simple enough, aim for 300 lines of code. The less codelines to maintain the better. I am fluent in python, no need to provide explanatory comments during generation, only drop a small inline comment during debugging, if the bug was not trivial. Assume senior software developer levels of knowledge, with 3+ years of python experience.
- KISS, not SOLID
- Use the attached domain objects

Description:

- The system has a rectangular shape measuring work area specified by points which coordinates are specified under "Rectangle" in `system_input_file.1630412935`.
- The robot arm, as part of the system, is moving around the working area checking points.

Requirements for system:

- System takes expected visited points from Input file (`system_input_file.1630412935.txt` "Points").
- System visits points one by one only within the work area specified by Rectangle.
- System provides the list of actual visited points in Output file (`system_output_file.1630412935.txt`).

Task at hand:

- Write tests script/provide a test automation solution that verifies the behavior of system based on requirements for system. The script should generate the test results file (`test_results.txt`) with the results of verification using following template:
- | Expected visited points | Actual visited points |
|---------------------------|-----------------------|
| Test result (PASS / FAIL) | |

 - Important! This output is highly underspecified, beyond having nice formatting to it, propose a solution, don't just blindly implement it.
- The results of verification is PASS when all of the 3 requirements are met.
 - This implies that a missing file also means a test fail. And in general, anomaly -> test fail, unless otherwise specified

- In case of FAIL log proper message in the console related to the failure. Be informative, but succinct.
- Naturally, the solution should be able to deal with different `system_input_file` and `system_output_file` too (architeturat support for separated filewriter functions).
 - This is trivial, but if there's another set of points are given by `system_input_file`, and different `system_output_file` is given by system, the test shall still be complete.
 - For this latter part, create on more input-output file pair with the existing format. These shall have the ending of `_testing_the_test` .

Harness:

- Use the coding environment harness. Automatically run your code, and verify that everything works.